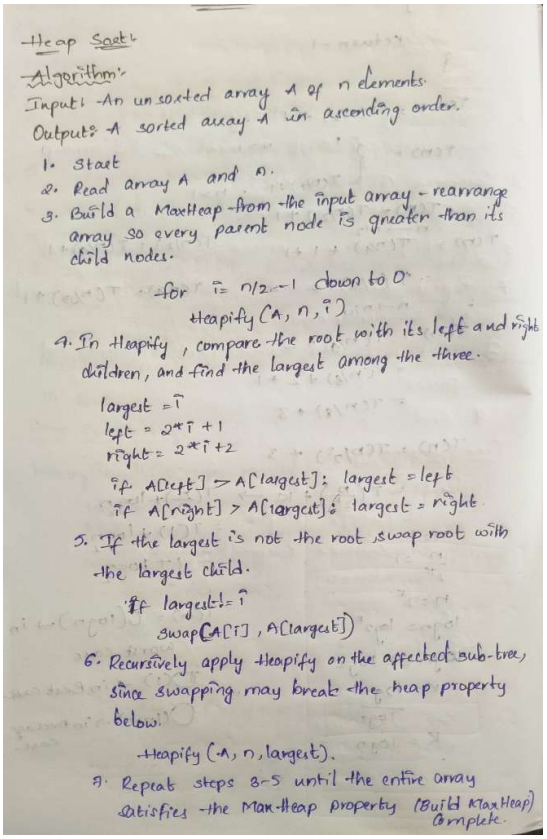
PRACTICAL: 03

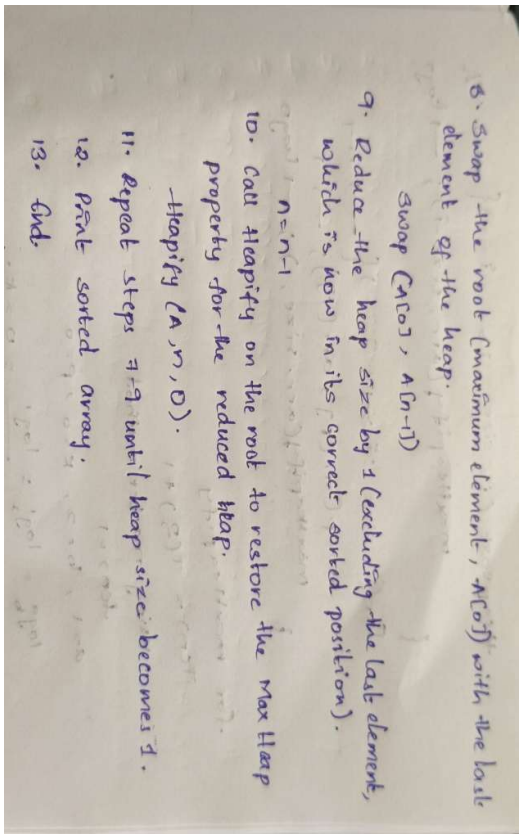
Aim: To implement the max-heap sorting algorithm.

Algorithm:



Heap Sort:
Algorithm:
Input: An unsorted array A of n elements.
Output: A sorted array A in ascending order.

1. Start
2. Read array A and n.
3. Build a MaxHeap from the input array - rearrange array so every parent node is greater than its child nodes.
   for i = n/2 - 1 down to 0
   Heapify (A, n, i)
4. In Heapify, compare the root with its left and right children, and find the largest among the three.
   largest = i
   left = 2*i + 1
   right = 2*i + 2
   if A[left] > A[largest]: largest = left
   if A[right] > A[largest]: largest = right
5. If the largest is not the root, swap root with the largest child.
   if largest != i
   swap(A[i], A[largest])
6. Recursively apply Heapify on the affected sub-tree, since swapping may break the heap property below.
   Heapify (A, n, largest).
7. Repeat steps 3-5 until the entire array satisfies the Max-Heap property (Build MaxHeap) Complete.



8. Swap the root (maximum element, A[0]) with the last element of the heap.
   swap (A[0], A[n-1])
9. Reduce the heap size by 1 (excluding the last element, which is now in its correct sorted position).
   n = n-1
10. Call Heapify on the root to restore the Max Heap property for the reduced heap.
    Heapify (A, n, 0).
11. Repeat steps 7-9 until heap size becomes 1.
12. Print sorted array.
13. End.

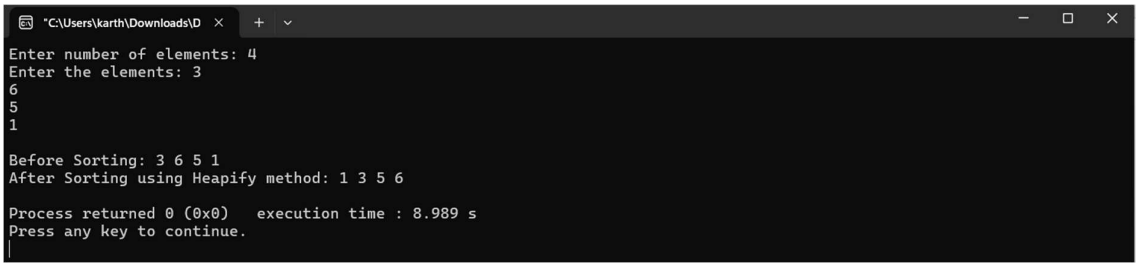
Program / Code:

```
#include <iostream>
using namespace std;
class MaxHeap {
public:
  int arr[100], n;
  void arrayInput() {
    cout << "Enter number of elements: ";
    cin >> n;
```

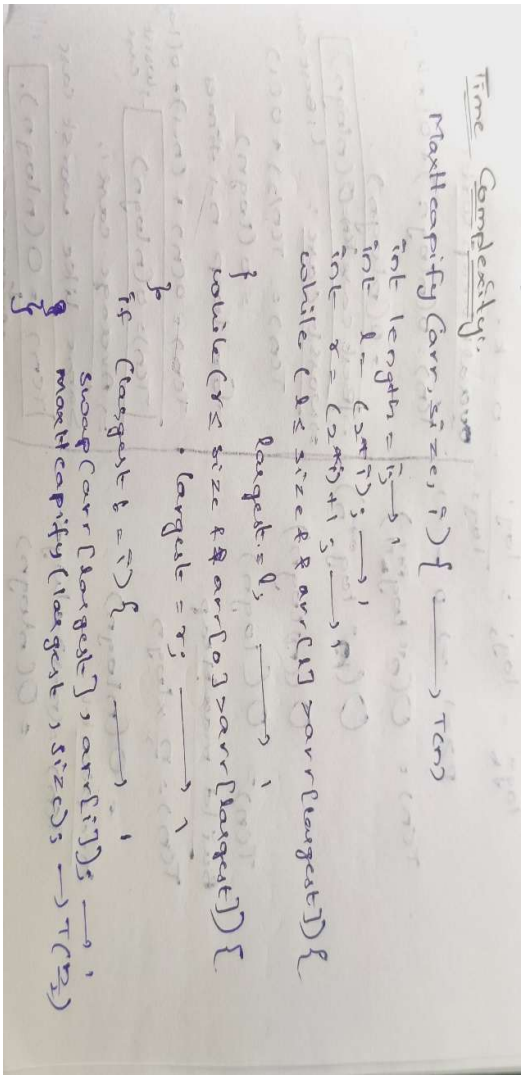
```
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}
void printArray() {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
void maxheapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }
    if (largest != i) {
        swap(arr[i], arr[largest]);
        maxheapify(arr, n, largest);
    }
}
void heapsort() {
    for (int i = n / 2 - 1; i >= 0; i--) {
        maxheapify(arr, n, i);
```

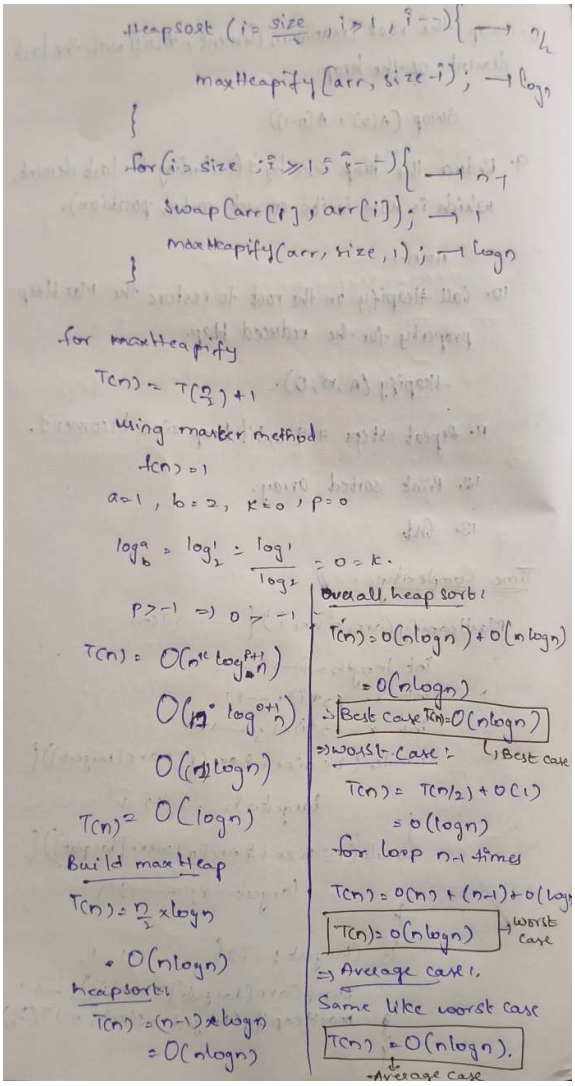
```
    }
    for (int i = n - 1; i > 0; i--) {
      swap(arr[0], arr[i]);
      maxheapify(arr, i, 0);
    }
  }
};
int main() {
  MaxHeap mh;
  mh.arrayInput();
  cout << "\nBefore Sorting: ";
  mh.printArray();
  mh.heapsort();
  cout << "After Sorting using Heapify method: ";
  mh.printArray();
  return 0;
}
```

Output:



Time Complexity:





Conclusion:

In this practical, we implemented the Max-Heap Sort algorithm successfully using the Max-Heapify method. The largest element is repeatedly placed at the end of the array, resulting in a sorted array in ascending order. The time complexity of Heap Sort is O(n log n).